# Queues: Complete Guide to All Implementations and Applications

## Introduction to Queue Data Structure

A **queue** is a fundamental linear data structure that follows the **First-In-First-Out (FIFO)** principle: the first element added to the queue is the first one to be removed. Think of a queue as a line of people waiting at a ticket counter—the person who arrives first is served first[143][155][^102]. This intuitive behavior makes queues essential for managing sequential processes, task scheduling, resource allocation, and algorithmic operations[145][148].

## Fundamental Terminology

Understanding queue terminology is crucial for mastering its operations[143][146]:

- **Front (Head)**: The position of the first element, which will be removed next during a dequeue operation
- **Rear (Tail)**: The position where new elements are added during enqueue operations
- **Size**: The current number of elements in the queue
- **Capacity**: The maximum number of elements the queue can hold (relevant for fixed-size implementations)
- **Enqueue**: Operation to add an element at the rear
- **Dequeue**: Operation to remove an element from the front
- **Overflow**: Attempting to enqueue into a full queue (fixed-size implementations)
- **Underflow**: Attempting to dequeue from an empty queue

## FIFO Principle in Detail

The FIFO (First-In-First-Out) principle is the defining characteristic of queues[143][155]. When elements are added to a queue, they form a sequential ordering. The element that has been in the queue longest is always the first to be removed. This contrasts with stacks (LIFO - Last-In-First-Out) and makes queues ideal for:

- **Order preservation**: Maintaining the sequence of arrival or request
- **Fair scheduling**: Ensuring resources are allocated in arrival order
- **Buffering**: Managing data streams where processing order matters
- **Breadth-first traversal**: Exploring data structures level by level

## Types of Queues

Queue data structures can be classified into several specialized variants, each optimized for specific use cases[143][152][^156]:

### 1. Simple Queue (Linear Queue)

A basic queue implementation following strict FIFO ordering:

- Insertion only at rear
- Deletion only from front
- Can be implemented using arrays or linked lists
- Array implementation may suffer from false overflow (unused space at beginning)[^143]

### 2. Circular Queue (Ring Buffer)

An optimized array implementation that treats the array as circular:

- Efficiently reuses memory by wrapping around
- Eliminates false overflow problem
- Uses modulo arithmetic for index calculation
- Ideal for fixed-size buffers in embedded systems and I/O management[147][153]

### 3. Priority Queue

Elements are assigned priorities; highest priority dequeued first:

- Does not follow strict FIFO if priorities differ
- Elements with equal priority follow FIFO
- Commonly implemented using heaps (binary heap, Fibonacci heap)
- Critical for task scheduling, graph algorithms (Dijkstra, Prim), and event simulation[150][162]

### 4. Double-Ended Queue (Deque)

Allows insertion and deletion from both ends:

- Combines features of both stack and queue
- Four primary operations: insertFront, insertRear, deleteFront, deleteRear
- Versatile for sliding window problems and palindrome checking
- Can function as both FIFO and LIFO structure[143][152][^156]

## Operations and Time Complexity

| Operation | Array Queue | Linked Queue | Circular Queue | Priority Queue (Heap) |
|---|---|---|---|---|
| Enqueue | O(1) | O(1) | O(1) | O(log n) |
| Dequeue | O(1)* | O(1) | O(1) | O(log n) |
| Peek/Front | O(1) | O(1) | O(1) | O(1) |
| isEmpty | O(1) | O(1) | O(1) | O(1) |
| isFull | O(1) | N/A | O(1) | O(1) |
| Search | O(n) | O(n) | O(n) | O(n) |

*Linear array queue may require O(n) shifting or suffer from false overflow

**Space Complexity**: O(n) for all implementations where n is the number of elements[152][156]

## Real-World Applications

Queues are ubiquitous in computing systems and algorithms[145][148]:

**Operating Systems:**

- Process scheduling (ready queue, job queue)
- Disk scheduling (I/O request management)
- Buffer management for I/O devices
- Interrupt handling

**Computer Networks:**

- Packet routing and buffering
- Message queuing systems (RabbitMQ, Apache Kafka)
- Network congestion management
- Quality of Service (QoS) implementation

**Algorithm Applications:**

- Breadth-First Search (BFS) in graphs
- Level-order traversal in trees
- Shortest path algorithms
- Cache implementation (LRU with deque)

**Software Systems:**

- Print spooling
- Asynchronous data transfer
- Call center systems

- Event handling in GUI applications

**Data Processing:**

- Stream processing pipelines
- Task queues in distributed systems
- Request handling in web servers
- Database transaction processing

## Advantages and Disadvantages

**Advantages:**

- Maintains ordered processing (fairness)
- Simple conceptual model
- Efficient O(1) enqueue and dequeue operations
- Natural fit for sequential processing

**Disadvantages:**

- No random access to middle elements
- Fixed size in array implementations (overflow risk)
- Linear array suffers from false overflow
- Priority queue has higher complexity (O(log n))

## Memory Considerations

**Array-Based Implementation:**

- Fixed size determined at creation
- Contiguous memory allocation
- Better cache locality for sequential access
- Risk of overflow or memory waste

**Linked List Implementation:**

- Dynamic size grows/shrinks as needed
- Non-contiguous memory (scattered in heap)
- No overflow risk (limited only by available memory)
- Additional memory overhead for pointers

**Circular Queue Optimization:**

- Best of both worlds for fixed-capacity scenarios
- Eliminates false overflow
- Efficient memory utilization

- Requires careful modulo arithmetic

Understanding these fundamental concepts provides the foundation for implementing and optimizing queue-based solutions in real-world applications and advanced algorithms.

# Array-Based Queue Implementation in C

### Simple Linear Queue

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;stdbool.h&gt;

#define MAX_SIZE 100

typedef struct {
    int data[MAX_SIZE];
    int front;
    int rear;
    int size;
} ArrayQueue;

// Initialize queue
void initQueue(ArrayQueue* q) {
    q-&gt;front = 0;
    q-&gt;rear = -1;
    q-&gt;size = 0;
}

// Check if queue is empty
bool isEmpty(ArrayQueue* q) {
    return q-&gt;size == 0;
}

// Check if queue is full
bool isFull(ArrayQueue* q) {
    return q-&gt;size == MAX_SIZE;
}

// Enqueue operation
bool enqueue(ArrayQueue* q, int value) {
    if (isFull(q)) {
        printf("Queue Overflow! Cannot enqueue %d\n", value);
        return false;
    }

    q-&gt;rear++;
    q-&gt;data[q-&gt;rear] = value;
    q-&gt;size++;
    return true;
}

// Dequeue operation
int dequeue(ArrayQueue* q) {
```

```
    if (isEmpty(q)) {
        printf("Queue Underflow! Cannot dequeue\n");
        return -1;
    }

    int value = q-&gt;data[q-&gt;front];
    q-&gt;front++;
    q-&gt;size--;

    // Reset pointers when queue becomes empty
    if (isEmpty(q)) {
        q-&gt;front = 0;
        q-&gt;rear = -1;
    }

    return value;
}

// Peek front element
int peek(ArrayQueue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty!\n");
        return -1;
    }
    return q-&gt;data[q-&gt;front];
}

// Display queue
void display(ArrayQueue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty!\n");
        return;
    }

    printf("Queue: ");
    for (int i = q-&gt;front; i &lt;= q-&gt;rear; i++) {
        printf("%d ", q-&gt;data[i]);
    }
    printf("\n");
}

// Get current size
int getSize(ArrayQueue* q) {
    return q-&gt;size;
}

int main() {
    ArrayQueue q;
    initQueue(&amp;q);

    printf("=== Simple Array Queue Demo ===\n\n");

    // Enqueue operations
    enqueue(&amp;q, 10);
    enqueue(&amp;q, 20);
    enqueue(&amp;q, 30);
```

```
    enqueue(&amp;q, 40);
    enqueue(&amp;q, 50);

    printf("After enqueuing 10, 20, 30, 40, 50:\n");
    display(&amp;q);
    printf("Size: %d\n\n", getSize(&amp;q));

    // Peek operation
    printf("Front element: %d\n\n", peek(&amp;q));

    // Dequeue operations
    printf("Dequeued: %d\n", dequeue(&amp;q));
    printf("Dequeued: %d\n\n", dequeue(&amp;q));

    printf("After dequeuing two elements:\n");
    display(&amp;q);
    printf("Size: %d\n", getSize(&amp;q));

    return 0;
}
```

### Notes on Simple Array Queue:

- **Limitation**: After multiple enqueue/dequeue operations, rear pointer reaches MAX_SIZE even if front positions are empty (false overflow)
- **Memory Waste**: Space before front index becomes unusable
- **Use Case**: Suitable only for one-time use or when queue is completely emptied periodically

# Circular Queue Implementation in C

### Optimized Array Queue Using Circular Logic

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;stdbool.h&gt;

#define MAX_SIZE 5

typedef struct {
    int data[MAX_SIZE];
    int front;
    int rear;
    int size;
    int capacity;
} CircularQueue;

// Initialize circular queue
void initCircularQueue(CircularQueue* q) {
    q-&gt;front = 0;
    q-&gt;rear = -1;
    q-&gt;size = 0;
```

```
    q-&gt;capacity = MAX_SIZE;
}

// Check if empty
bool isEmptyCircular(CircularQueue* q) {
    return q-&gt;size == 0;
}

// Check if full
bool isFullCircular(CircularQueue* q) {
    return q-&gt;size == q-&gt;capacity;
}

// Enqueue with circular wraparound
bool enqueueCircular(CircularQueue* q, int value) {
    if (isFullCircular(q)) {
        printf("Circular Queue Overflow! Cannot enqueue %d\n", value);
        return false;
    }

    // Circular increment using modulo
    q-&gt;rear = (q-&gt;rear + 1) % q-&gt;capacity;
    q-&gt;data[q-&gt;rear] = value;
    q-&gt;size++;

    printf("Enqueued %d at position %d\n", value, q-&gt;rear);
    return true;
}

// Dequeue with circular logic
int dequeueCircular(CircularQueue* q) {
    if (isEmptyCircular(q)) {
        printf("Circular Queue Underflow! Cannot dequeue\n");
        return -1;
    }

    int value = q-&gt;data[q-&gt;front];
    printf("Dequeued %d from position %d\n", value, q-&gt;front);

    q-&gt;front = (q-&gt;front + 1) % q-&gt;capacity;
    q-&gt;size--;

    return value;
}

// Peek front
int peekCircular(CircularQueue* q) {
    if (isEmptyCircular(q)) {
        printf("Circular Queue is empty!\n");
        return -1;
    }
    return q-&gt;data[q-&gt;front];
}

// Display circular queue
void displayCircular(CircularQueue* q) {
```

```
    if (isEmptyCircular(q)) {
        printf("Circular Queue is empty!\n");
        return;
    }

    printf("Circular Queue: ");
    int count = 0;
    int index = q-&gt;front;

    while (count &lt; q-&gt;size) {
        printf("%d ", q-&gt;data[index]);
        index = (index + 1) % q-&gt;capacity;
        count++;
    }
    printf("\n");
    printf("Front: %d, Rear: %d, Size: %d/%d\n",
           q-&gt;front, q-&gt;rear, q-&gt;size, q-&gt;capacity);
}

int main() {
    CircularQueue cq;
    initCircularQueue(&amp;cq);

    printf("=== Circular Queue Demo ===\n\n");
    printf("Queue Capacity: %d\n\n", MAX_SIZE);

    // Fill the queue
    enqueueCircular(&amp;cq, 10);
    enqueueCircular(&amp;cq, 20);
    enqueueCircular(&amp;cq, 30);
    enqueueCircular(&amp;cq, 40);
    enqueueCircular(&amp;cq, 50);
    printf("\n");
    displayCircular(&amp;cq);

    // Try to enqueue when full
    printf("\nAttempting to enqueue when full:\n");
    enqueueCircular(&amp;cq, 60);

    // Dequeue some elements
    printf("\nDequeuing 2 elements:\n");
    dequeueCircular(&amp;cq);
    dequeueCircular(&amp;cq);
    printf("\n");
    displayCircular(&amp;cq);

    // Now we can enqueue again (circular reuse)
    printf("\nEnqueuing new elements (circular wraparound):\n");
    enqueueCircular(&amp;cq, 60);
    enqueueCircular(&amp;cq, 70);
    printf("\n");
    displayCircular(&amp;cq);

    return 0;
}
```

### Key Advantages of Circular Queue:

- **Memory Efficiency**: Reuses space freed by dequeue operations
- **No False Overflow**: Can utilize full capacity
- **Constant Time**: O(1) for all operations
- **Perfect for Buffers**: Ideal for streaming data, producer-consumer scenarios

# Linked List Queue Implementation in C

### Dynamic Queue with No Size Limit

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;stdbool.h&gt;

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct {
    Node* front;
    Node* rear;
    int size;
} LinkedQueue;

// Create new node
Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) {
        printf("Memory allocation failed!\n");
        return NULL;
    }
    newNode-&gt;data = value;
    newNode-&gt;next = NULL;
    return newNode;
}

// Initialize linked queue
void initLinkedQueue(LinkedQueue* q) {
    q-&gt;front = NULL;
    q-&gt;rear = NULL;
    q-&gt;size = 0;
}

// Check if empty
bool isEmptyLinked(LinkedQueue* q) {
    return q-&gt;front == NULL;
}

// Enqueue operation
bool enqueueLinked(LinkedQueue* q, int value) {
```

```
    Node* newNode = createNode(value);
    if (!newNode) return false;

    // If queue is empty, both front and rear point to new node
    if (isEmptyLinked(q)) {
        q-&gt;front = q-&gt;rear = newNode;
    } else {
        // Add new node at rear
        q-&gt;rear-&gt;next = newNode;
        q-&gt;rear = newNode;
    }

    q-&gt;size++;
    printf("Enqueued: %d\n", value);
    return true;
}

// Dequeue operation
int dequeueLinked(LinkedQueue* q) {
    if (isEmptyLinked(q)) {
        printf("Queue Underflow! Cannot dequeue\n");
        return -1;
    }

    Node* temp = q-&gt;front;
    int value = temp-&gt;data;

    q-&gt;front = q-&gt;front-&gt;next;

    // If queue becomes empty, update rear
    if (q-&gt;front == NULL) {
        q-&gt;rear = NULL;
    }

    free(temp);
    q-&gt;size--;

    printf("Dequeued: %d\n", value);
    return value;
}

// Peek front
int peekLinked(LinkedQueue* q) {
    if (isEmptyLinked(q)) {
        printf("Queue is empty!\n");
        return -1;
    }
    return q-&gt;front-&gt;data;
}

// Display queue
void displayLinked(LinkedQueue* q) {
    if (isEmptyLinked(q)) {
        printf("Queue is empty!\n");
        return;
    }
```

```
    printf("Linked Queue: ");
    Node* current = q-&gt;front;
    while (current != NULL) {
        printf("%d ", current-&gt;data);
        current = current-&gt;next;
    }
    printf("\nSize: %d\n", q-&gt;size);
}

// Free entire queue
void freeQueue(LinkedQueue* q) {
    while (!isEmptyLinked(q)) {
        dequeueLinked(q);
    }
}

int main() {
    LinkedQueue lq;
    initLinkedQueue(&amp;lq);

    printf("=== Linked List Queue Demo ===\n\n");

    // Enqueue operations
    enqueueLinked(&amp;lq, 100);
    enqueueLinked(&amp;lq, 200);
    enqueueLinked(&amp;lq, 300);
    enqueueLinked(&amp;lq, 400);
    enqueueLinked(&amp;lq, 500);
    printf("\n");
    displayLinked(&amp;lq);

    printf("\nFront element: %d\n\n", peekLinked(&amp;lq));

    // Dequeue operations
    dequeueLinked(&amp;lq);
    dequeueLinked(&amp;lq);
    printf("\n");
    displayLinked(&amp;lq);

    // More enqueues
    printf("\nAdding more elements:\n");
    enqueueLinked(&amp;lq, 600);
    enqueueLinked(&amp;lq, 700);
    printf("\n");
    displayLinked(&amp;lq);

    // Cleanup
    printf("\nFreeing queue...\n");
    freeQueue(&amp;lq);
    displayLinked(&amp;lq);

    return 0;
}
```

### Advantages of Linked Queue:

- **Dynamic Size**: No fixed capacity, grows as needed
- **No Overflow**: Limited only by available memory
- **Efficient Memory Use**: Only allocates what's needed

# Priority Queue Implementation in C

### Min-Heap Based Priority Queue

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;stdbool.h&gt;
#include &lt;limits.h&gt;

#define MAX_SIZE 100

typedef struct {
    int data;
    int priority;
} PQElement;

typedef struct {
    PQElement heap[MAX_SIZE];
    int size;
} PriorityQueue;

// Initialize priority queue
void initPriorityQueue(PriorityQueue* pq) {
    pq-&gt;size = 0;
}

// Check if empty
bool isEmptyPQ(PriorityQueue* pq) {
    return pq-&gt;size == 0;
}

// Check if full
bool isFullPQ(PriorityQueue* pq) {
    return pq-&gt;size == MAX_SIZE;
}

// Swap two elements
void swap(PQElement* a, PQElement* b) {
    PQElement temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify up (for insertion)
void heapifyUp(PriorityQueue* pq, int index) {
    while (index &gt; 0) {
        int parent = (index - 1) / 2;
```

```
        // Min-heap: parent should have lower priority value
        if (pq-&gt;heap[parent].priority &gt; pq-&gt;heap[index].priority) {
            swap(&amp;pq-&gt;heap[parent], &amp;pq-&gt;heap[index]);
            index = parent;
        } else {
            break;
        }
    }
}

// Heapify down (for deletion)
void heapifyDown(PriorityQueue* pq, int index) {
    int size = pq-&gt;size;

    while (index &lt; size) {
        int smallest = index;
        int left = 2 * index + 1;
        int right = 2 * index + 2;

        // Find smallest among node and its children
        if (left &lt; size &amp;&amp; pq-&gt;heap[left].priority &lt; pq-&gt;heap[smalles
            smallest = left;
        }

        if (right &lt; size &amp;&amp; pq-&gt;heap[right].priority &lt; pq-&gt;heap[smal]
            smallest = right;
        }

        if (smallest != index) {
            swap(&amp;pq-&gt;heap[index], &amp;pq-&gt;heap[smallest]);
            index = smallest;
        } else {
            break;
        }
    }
}

// Insert element with priority
bool enqueuePQ(PriorityQueue* pq, int data, int priority) {
    if (isFullPQ(pq)) {
        printf("Priority Queue Overflow!\n");
        return false;
    }

    // Add element at end
    pq-&gt;heap[pq-&gt;size].data = data;
    pq-&gt;heap[pq-&gt;size].priority = priority;

    // Heapify up to maintain heap property
    heapifyUp(pq, pq-&gt;size);
    pq-&gt;size++;

    printf("Enqueued: data=%d, priority=%d\n", data, priority);
    return true;
}
```

```
// Remove and return highest priority element
int dequeuePQ(PriorityQueue* pq) {
    if (isEmptyPQ(pq)) {
        printf("Priority Queue Underflow!\n");
        return -1;
    }

    // Root has highest priority (lowest priority value in min-heap)
    int data = pq-&gt;heap[^0].data;
    int priority = pq-&gt;heap[^0].priority;

    // Move last element to root
    pq-&gt;heap[^0] = pq-&gt;heap[pq-&gt;size - 1];
    pq-&gt;size--;

    // Heapify down to maintain heap property
    if (pq-&gt;size &gt; 0) {
        heapifyDown(pq, 0);
    }

    printf("Dequeued: data=%d, priority=%d\n", data, priority);
    return data;
}

// Peek highest priority element
int peekPQ(PriorityQueue* pq) {
    if (isEmptyPQ(pq)) {
        printf("Priority Queue is empty!\n");
        return -1;
    }
    return pq-&gt;heap[^0].data;
}

// Display priority queue
void displayPQ(PriorityQueue* pq) {
    if (isEmptyPQ(pq)) {
        printf("Priority Queue is empty!\n");
        return;
    }

    printf("Priority Queue (heap order):\n");
    for (int i = 0; i &lt; pq-&gt;size; i++) {
        printf("  [Data: %d, Priority: %d]\n",
               pq-&gt;heap[i].data, pq-&gt;heap[i].priority);
    }
    printf("Size: %d\n", pq-&gt;size);
}

int main() {
    PriorityQueue pq;
    initPriorityQueue(&amp;pq);

    printf("=== Priority Queue Demo (Min-Heap) ===\n");
    printf("Lower priority value = Higher priority\n\n");
```

```
    // Enqueue with different priorities
    enqueuePQ(&amp;pq, 100, 3);
    enqueuePQ(&amp;pq, 200, 1);  // Highest priority
    enqueuePQ(&amp;pq, 300, 5);
    enqueuePQ(&amp;pq, 400, 2);
    enqueuePQ(&amp;pq, 500, 4);

    printf("\n");
    displayPQ(&amp;pq);

    // Dequeue - should come out in priority order
    printf("\nDequeuing in priority order:\n");
    while (!isEmptyPQ(&amp;pq)) {
        dequeuePQ(&amp;pq);
    }

    return 0;
}
```

### Priority Queue Characteristics:

- **Heap-Based**: O(log n) insertion and deletion
- **Priority Ordering**: Elements dequeued by priority, not insertion order
- **Applications**: Task scheduling, Dijkstra's algorithm, Huffman coding, event simulation

# Advanced Applications and Algorithms

### Application 1: Breadth-First Search (BFS)

```
#include &lt;stdio.h&gt;
#include &lt;stdbool.h&gt;

#define MAX_VERTICES 10

// Simple queue for BFS
typedef struct {
    int items[MAX_VERTICES];
    int front, rear;
} Queue;

void initQueue(Queue* q) {
    q-&gt;front = -1;
    q-&gt;rear = -1;
}

bool isEmpty(Queue* q) {
    return q-&gt;front == -1;
}

void enqueue(Queue* q, int value) {
    if (q-&gt;rear == MAX_VERTICES - 1) return;
```

```
    if (q-&gt;front == -1) q-&gt;front = 0;
    q-&gt;rear++;
    q-&gt;items[q-&gt;rear] = value;
}

int dequeue(Queue* q) {
    if (isEmpty(q)) return -1;

    int item = q-&gt;items[q-&gt;front];
    if (q-&gt;front &gt;= q-&gt;rear) {
        q-&gt;front = q-&gt;rear = -1;
    } else {
        q-&gt;front++;
    }
    return item;
}

// BFS traversal
void BFS(int graph[MAX_VERTICES][MAX_VERTICES], int vertices, int start) {
    bool visited[MAX_VERTICES] = {false};
    Queue q;
    initQueue(&amp;q);

    visited[start] = true;
    enqueue(&amp;q, start);

    printf("BFS Traversal starting from vertex %d: ", start);

    while (!isEmpty(&amp;q)) {
        int current = dequeue(&amp;q);
        printf("%d ", current);

        // Explore all adjacent vertices
        for (int i = 0; i &lt; vertices; i++) {
            if (graph[current][i] == 1 &amp;&amp; !visited[i]) {
                visited[i] = true;
                enqueue(&amp;q, i);
            }
        }
    }
    printf("\n");
}

int main() {
    int vertices = 6;
    int graph[MAX_VERTICES][MAX_VERTICES] = {
        {0, 1, 1, 0, 0, 0},
        {1, 0, 0, 1, 1, 0},
        {1, 0, 0, 0, 1, 0},
        {0, 1, 0, 0, 0, 1},
        {0, 1, 1, 0, 0, 1},
        {0, 0, 0, 1, 1, 0}
    };

    printf("=== BFS Application ===\n");
    BFS(graph, vertices, 0);
```

```
    return 0;
}
```

## Summary: Queue Types Comparison

| Feature | Simple Array | Circular Queue | Linked List | Priority Queue |
|---|---|---|---|---|
| **Implementation** | Static array | Static array + modulo | Dynamic nodes | Heap (array/tree) |
| **Size** | Fixed | Fixed | Dynamic | Fixed or Dynamic |
| **Memory** | Contiguous | Contiguous | Scattered | Contiguous (heap) |
| **Enqueue** | O(1) | O(1) | O(1) | O(log n) |
| **Dequeue** | O(1) | O(1) | O(1) | O(log n) |
| **Overflow Risk** | Yes (false) | Yes | No | Yes (if array) |
| **Memory Efficiency** | Poor | Excellent | Good | Good |
| **Use Case** | Simple, temporary | Buffers, streaming | General purpose | Scheduling, algorithms |

This comprehensive guide covers all major queue implementations in C, from basic concepts to advanced applications, providing the foundation for mastering queue-based algorithms and system design.
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21]

⁂


1. https://www.geeksforgeeks.org/dsa/introduction-to-queue-data-structure-and-algorithm-tutorials/
2. https://www.thedshandbook.com/queues/
3. https://www.geeksforgeeks.org/dsa/circular-linked-list-implementation-of-circular-queue/
4. https://docs.oracle.com/database/121/ADQUE/aq_intro.htm
5. https://www.w3schools.com/dsa/dsa_data_queues.php
6. https://simplerize.com/data-structures/queue-data-structure
7. https://sic.ici.ro/vol-14-no-2-2005/advanced-queue-management-algorithms-for-computer-networks/
8. https://www.youtube.com/watch?v=zp6pBNbUB2U
9. https://prepinsta.com/c-program/circular-queue-using-linked-list/
10. https://dzone.com/articles/oracle-advanced-queue-a-guide
11. https://www.geeksforgeeks.org/dsa/queue-data-structure/
12. https://stackoverflow.com/questions/61824879/an-efficient-way-to-implement-circular-priority-queue
13. https://anandgharu.wordpress.com/wp-content/uploads/2023/11/unit-6-fds-2023.pdf
14. https://www.geeksforgeeks.org/dsa/introduction-to-queue-data-structure-and-algorithm-tutorials/
15. https://www.geeksforgeeks.org/dsa/applications-of-queue-data-structure/
16. https://www.simplilearn.com/tutorials/data-structure-tutorial/queue-in-data-structure
17. https://www.analytixlabs.co.in/blog/circular-queue-in-data-structure/

18. https://www.iquanta.in/blog/top-10-applications-of-queue-in-data-structure-in-2025/
19. https://logicmojo.com/data-structures-queue
20. https://www.uobabylon.edu.iq/eprints/publication_5_8293_1456.pdf
21. https://www.wavetec.com/blog/queue-management/advanced-techniques/